

Abstract

Computational generative grammar is a branch of modern linguistics stipulating that all theories and hypotheses must be justified by showing that the data follows from the theory. Due to the adverse complexity profile this level of rigor can only be reached with the aid of computational techniques.

Keywords: generative grammar; computation; computational generative grammar

1. Introduction

The goal of linguistic theory is to describe and explain a set of observations (“data”). The theory is justified to the extent it satisfies this condition. How to verify if it does?

In the natural sciences, to begin with something uncontroversial, part of the justification reduces to an unambiguous formal sequence of operations – that is, calculations – which demonstrates that the facts follow from the theory. This has constituted a minimal requirement for justification in all advanced sciences since the 17th century. Let us say that the theory has a *proper justification* only if it comes with a justification in this sense.¹

Proper justification is virtually absent from modern linguistics. Most theories and theoretical claims are provisional in the sense that we do not know if any of them describes let alone explains the subject matter facts. Some linguistic theories never describe what the totality of the intended facts are (this applies above all to generative theorizing); most theories are expressed in an ambiguous

¹ Formal proof is a necessary but not sufficient criterion for justification. Several other attributes such as elegance and simplicity matter as well. I will return to them later in this article.

language (all informal and pseudo-formal grammars); and almost no theory or hypothesis has been justified by showing that the facts of interest (when they are provided in unambiguous format) follow from the theory (if described unambiguously).

Conversely, in order to justify a linguistic theory the following three conditions must be met: (i) the totality of the facts of interest (“data”) must exist in an unambiguous form; (ii) the theory must be provided in an unambiguous language; (iii) there must exist a sequence of formal operations which shows that (i) follows from (ii). Most linguistic theories fail all three conditions.

A theory that lacks proper justification in the above sense must at some point rely on subjective opinions provided by the stakeholders, such as the author(s). In the absence of proper justification the stakeholders substitute rigorous proofs with ordinary language notions such as “it is obvious that,” “it follows” or “the data shows.” In essence they imagine a sequence of inferential steps connecting the theory with data.

The problem with human imagination is not that it cannot produce useful or true theories. An intuitive idea can provide a useful basis for rigorous theory development. Human intuition plays a role in scientific discovery. Newton relied on geometrical intuitions and imagination. But discovery is not justification. The problem is that a theory without proof can also be refuted without proof; as much as opinions can be useful they can also be junk. Proper justification removes these uncertainties.² Newton’s scientific work is almost in its entirety made up of proofs, not opinions.

2. Proper justification in linguistics

The methodological requirement we are now discussing is that the data must follow from the theory. Moreover, there must exist an unambiguous demonstration that it does. Since the demonstration

2 I have discussed these methodological concerns in several papers, see for example Brattico (2019, 2021a, 2021b, submitted). There are also several video presentations discussing the method, see <https://www.youtube.com/@paulibrattico6152>. Currently there are three lecture courses: one basic course in English (<https://www.youtube.com/playlist?list=PL35upitLda1eQeoEit7yGO6HvOGeKGHri>), two in Finnish (<https://www.youtube.com/playlist?list=PL35upitLda1fkGLFrDYEEy5P1LBq74dZ3>, <https://www.youtube.com/playlist?list=PLUfVLIHh2hDM>), although it should be possible to view the latter with English translations. More material is to come.

must be unambiguous, there must exist a procedure that ‘replicates the data’. Much like a physical law allows one to replicate measurements by calculations, in linguistics we must replicate language data by utilizing calculations from the theory. This presupposes that the data exists in an unambiguous form.

In a typical linguistic context the data consist of expressions in some language or several plus their properties, whatever is of interest in the particular study. The properties can be meanings, binding dependencies, pragmatic properties or contexts, ambiguities, prosodic features, processing-related measurements, or properties such as grammaticality or acceptability. In short the dataset must contain objective, replicable and systematic measurements of human linguistic behavior. We will return to the details in Section 4 and assume for the time being that we have such dataset at hand.

Let us now imagine a linguistic theory that tries to predict for example the topic or focus interpretations for a range of expressions in some language such as Finnish, of which we have a dataset available.³ The theory could process each sentence in the dataset and predict the segments native speakers interpret as the topic and focus elements, and then compare the output with the attested behavior that has been recorded into the dataset. This type of theory constitutes a *recognition grammar*, because it recognizes (e.g., detects, infers, processes) properties of the input expressions, the latter which are provided as input. Technically a recognition grammar defines a set of expressions and their properties by providing a function from the former into latter. A function that rejects or accepts expressions together with their properties defines a *characteristic function* for the set. We can, in other words, define and thus replicate the dataset by recognition, ultimately by means of implementing its characteristic function.

We can accomplish the same outcome by enumeration. An *enumerative grammar* defines the set (that is, our dataset containing expressions and their properties) by listing its members one by one. Therefore in a study focusing on the topic and focus interpretations an enumerative grammar would

3 I will use the terms “grammatical theory,” “grammatical hypothesis” and “grammar” interchangeably in this article, though they have slightly different uses and connotations.

generate the relevant expressions together with their interpretations. A particular type of enumerative grammar works by generating a set of grammatical or attested expressions and only them.⁴ For example, consider a rule which pairs transitive verbs with direct arguments (e.g., *admires* + *Mary*). Because the grammar cannot combine transitive verbs directly with their subjects, sentences such as **John admires* are never generated and are, on such basis, predicted to be ungrammatical or unattested. Hence in this type of approach what is not generated is predicted to be unacceptable, ungrammatical or otherwise unattested.

Recognition and enumerative grammars are mathematically equivalent under very weak conditions, and from an empirical point of view one can usually use either one.⁵ But because both recognition and enumeration can define sets of expressions, they are *generative grammars* in the technical sense the term “generative” was used in the 1950s and 1960s. The term “generative grammar” in the title of this article refers to generative grammars in this sense.⁶

Any reasonable linguistic theory must contain a generative engine that defines larger expressions by composing them from parts such as words. *John admires Mary* is composed in some way from its constituent expressions *John*, *admires* and *Mary*. Let us say that the creation of an expression by the application of the rules of the grammar is called its *derivation*. To show that the theory replicates the properties of some one expression in the dataset we must construct a derivation for that expression. We can write down the required derivation and verify that the output matches with what we have in the dataset. Much of the linguistic literature consists of attempts at sketching and/or imagining such proofs.

4 We could work out an enumerative grammar that generates also ungrammatical expressions but marks them as such. This could be useful if we wanted to predict graded acceptability judgments.

5 Recognition grammars are perhaps more ‘comprehension-oriented’ than enumerative grammars, but it is up to the theory or hypothesis to determine its relation to actual comprehension and production. Most grammars and grammatical frameworks remain agnostic.

6 The term “generative” in the title of this article denotes generation in the technical sense; it does not refer to any particular grammatical theory, such as minimalism, developed within the ‘generative grammar’ framework. The generative grammars available today in the published literature are not generative in this sense, since they do not generate anything.

Up to this point everything seems doable: we could, at least in theory, substitute sketches of derivations with the real ones, and therefore remove opinions and guesswork from justifications. The problem is that we must also show that the grammar does not produce unattested expressions. To show that the theory does not generate unattested expressions and/or unattested properties we have to show that there is *no* derivation which produces such things. To show this, then, one has to explore all possible derivations. A paper-and-pencil demonstration of this type would consume years; the amount of derivational steps is simply too large.

The same problem emerges with linguistic ambiguity. Sentence *John saw Mary with a telescope* can be interpreted in one of two ways depending on who has the telescope. Therefore it is not sufficient to write down one derivation; we must explore all derivations to find ambiguities, and moreover we must verify that their properties too are predicted correctly.

So we cannot avoid calculating through all derivations. On the other hand, the number of calculation steps is too large. It seems we are stuck with impossible demands.

Calculations that are too demanding for human calculators are often trivial for computers. Instead of drafting the derivations by paper and pencil we propose to create a recursive function which explores the derivations for us. This motivates the term “computational” in the title of this article. Therefore the term “computational generative grammar” mentioned in the title of this article refers to a type of generative grammar that uses computational techniques to show that the grammar (grammatical theory, hypothesis) generates the attested expressions and their properties and only those. Let us construct the derivational search function.

3. Derivational search function

As stated in the preceding section, a derivation is a sequence of operations yielding expressions and their properties as its output. Let us assume that the derivation combines words into large phrasal units.⁷ Between the first input and final output there exists a sequence of states which represent the

7 The observation that phonological words do not constitute necessarily the real primitives of the human grammar does not matter here since the derivational search function could be applied to

intermediate results. If the input set is $\{John, admires, Mary\}$ and the output $\{John admires Mary\}$, we could have an intermediate step $\{John, admires Mary\}$ generated by concatenating *admires* and *Mary*. Let us assume that all these sets are *working space configurations* which represent the output of all previous applications of the operations. Moreover, at each stage we will apply some grammatical operation (or several, ignored for now) which will produce a new working space configuration. For example if the initial working space configuration is $\{Mary, admires, John\}$, we could concatenate *admires* + *Mary* and produce $\{John, admires Mary\}$. We could also have generated $\{Mary, admires John\}$. As a matter of fact, and as pointed out above, we must explore *every* derivation generated by the grammar. To do this computationally we use the following recursive algorithm⁸:

Derivational search function

Step 1. Assume a working space configuration (WSC, set of expressions) as input;

Step 2a. if the derivation can no longer continue, send it to the spell-out module; otherwise

Step 2b(i) for all operations OP_n of the theory, n = the number of input items,

(ii) for all n -tuples $\langle x_1 \dots x_n \rangle$ of items WSC,

(iii) Apply $OP(x_1 \dots x_n)$, create new WSC, and recurse into Step 1.

sublexical items such as morphemes, inflectional suffixes and clitics. Moreover, I will simplify the procedure by working with concrete phonological words and phrases; in a realistic grammar we would work with abstract linguistic representations such as phrase structure objects which produce the final spell-out forms by linearization and other possible postsyntactic operations. See Appendix B.

⁸ Recursive branching means that the state of the current working space is stored into memory before the same function is executed with the new working space configuration. Each function call therefore has its own memory state. When the executive control is returned back to the caller it can continue working with the configuration that existed before the call.

This function explores all derivations defined by the grammar.⁹ We demonstrate how it works below. However, the function presupposes that we have unambiguous representations for the working space configurations and operations OP. In a realistic grammar the working space configurations will likely contain abstract linguistic objects such as phrase structure trees, and there are several assembly operations such as Merge, Move and Agree. Before we can test and apply the derivational search function these components must be provided in some form.¹⁰ How to do this in Python is discussed in Appendix B.

The above derivational search function works best with enumerative grammars. Recognition grammars might rely on a slightly different function that we will ignore here, but the basic idea is the same. See Brattico (2019).

Let us consider the derivational search function used in the study Brattico (submitted) that was concerned with relative clauses in Finnish and English.¹¹ The dataset¹² contained a list of *data items*, each being made out of a set of initial lexical elements (the *numeration*) followed by a list of *target expressions*. The script created all possible derivations from the numeration and matched the

9 The above formulation provides only the bare essentials. In the published literature we find several additional features. For example, application of the operations OP is usually blocked by preconditions which reject derivations that cannot produce grammatical operations. Checking is performed before the recursive call (2b(iii)). We could block derivations on the basis of lexical subcategorization. The spell-out module referred to in 2a usually contains additional well-formedness filters applied to finished derivational outputs. It may contain additional operations that have to do with postsyntactic processing, such as clitic attachment, morphological adjustments and others, depending on the theory. There might be logging functions which store the derivational steps into an output file. Semantic and pragmatic interpretations can be generated for finished derivations by using additional interpretative functions. If the derivations start off from semantic units instead of words, then the semantic representations are part of the working space and the linguistic representations are derived during 2a. Linearization is part of 2a unless it involves derivational branching i.e. the system can attempt several linearization solutions for any given numeration.

10 Implementation of 2b(i) in such a way that it allows one to develop the theory without making any changes to the function itself requires that there is some systematic higher-order representation for the notion of ‘operation of the grammar’. In Python this can be done by defining the operations as functions and by creating a list of the said functions. See Appendix A.

11 The source code for this project which also replicates what is reported in this article can currently be found from <https://github.com/pajubrat/Study-21-relative-clause>.

12 <https://github.com/pajubrat/Study-21-relative-clause/blob/main/dataset.txt>

outcome with the target expressions. For example, consider the following (extremely simplistic) data item:

```
35  # Transitive VP
36
37  Numeration=the,man,bites*,the,dog
38  the man bites* the dog
39  the dog bites* the man
```

Line 35 is a comment. Line 37 provides a list of initial lexical items.¹³ The lexical items were generated on the basis of the lexicon. *bites** is an artificial transitive verb used to test bare verb phrases. The two target sentences (lines 38, 39) then follow. The computational script checks that the grammar produces these items and nothing else. Notice that the above item is not meant as part of a realistic or interesting grammar; it was used in this study to check in a simplified and controlled environment that lexical selection works as intended.

How can we know what the derivational search function does? We know this because the function stores its operations into a *log file*. The first acceptable expression was generated by the following steps:

```
582  1.  { the, the, man, dog, bites* }
583      MERGE(the, man) =
584      { the, dog, bites*, [DP the man] }
585
586  2.  { the, dog, bites*, [DP the man] }
587      MERGE(bites*, [DP the man]) =
588      { the, dog, [VP bites* [DP the man]] }
589
590  3.  { the, dog, [VP bites* [DP the man]] }
591      MERGE(the, dog) =
592      { [VP bites* [DP the man]], [DP the dog] }
593
594  4.  { [VP bites* [DP the man]], [DP the dog] }
595      MERGE([DP the dog], [VP bites* [DP the man]]) =
596      { [VP [DP the dog] [VP bites* [DP the man]]] }
597      FEATURES= the: {[D][COMP:N][LANG:EN]}      dog: {[LANG:EN]}
598  => /the dog bites* the man/ accepted.
```

The first line represents the working space configuration when the derivational search function was called for the first time, and contains a set of initial lexical items. The first operation was Merge which combined the definite article *the* with the noun head *man* (line 583). The third line (line 588)

13 The items appearing here are string literals that are matched with lexical items stored in the lexicon. We ignore this process here.

shows the new working space configuration, which was used as input for the first recursive call (line 586). After generating the first acceptable output *the dog bites* the man* (line 598) the function backtracks and generates *the man bites* the dog*:

```

603  5.  { the, dog, bites*, [DP the man] }
604      MERGE(the, dog) =
605      { bites*, [DP the man], [DP the dog] }
606
607  6.  { bites*, [DP the man], [DP the dog] }
608      MERGE(bites*, [DP the dog]) =
609      { [VP bites* [DP the dog]], [DP the man] }
610
611  7.  { [VP bites* [DP the dog]], [DP the man] }
612      MERGE([DP the man], [VP bites* [DP the dog]]) =
613      { [VP [DP the man] [VP bites* [DP the dog]]] }
614      FEATURES= the: {[D][COMP:N][LANG:EN]}      man: {[LANG
615 => /the man bites* the dog/ accepted.

```

Line 603 provides the working space configuration after backtracking. It tries a derivation in which *the man* and *the dog* are merged around the verb in a different order (line 615). After 36 steps the derivational search function had exhausted the derivations that the grammar generates from this numeration.¹⁴ We now have an unambiguous demonstration in the sense of Section 1 that the grammar deduces the properties of this data item.

The demonstration exists in the log file containing the steps required to show that (i) the two target sentences follow from the grammar and (ii) no other sentences follow. When the method is applied successfully to the whole dataset – an automatized procedure we can add to the script¹⁵ – a demonstration comes into existence which shows that the data follows from the theory. Hence we can now say that *the theory was justified by the data*. Approximately two million calculation steps were required to justify the theory proposed in this study – too many for any conceivable paper-and-pencil method.

14 The search space was pruned by limiting Merge to combinations that make empirical sense. For example, Merge(*the*, *the*) is considered, rejected and never explored further. Notice that by convention this step is not reported in the log file.

15 https://github.com/pajubrat/Study-21-relative-clause/blob/main/derive_relative_clauses.py

Existence of proper justification removes subjective opinion from the argument. The skeptic is no longer able to refute the theory by expressing a contrary opinion. The skeptic can still present three challenges.

The first possible concern is if there are calculation errors that the method did not reveal. For example, the script can generate correct outputs for wrong reasons. The skeptic can point out these errors and then either produce a more correct demonstration or show that the errors require minor or major rethinking of the hypothesis. Minimally, the skeptic can challenge the author to present an error-free proof. Occasional calculation errors are an inevitable side-effect of this method, especially when the calculations themselves become more complex.

The second option is to show that there exist additional data that the theory cannot deduce, and then construct a better theory deducing the new and the old data. The missing data must be relevant to the topic at hand, an issue to which we return in Section 4. But the skeptic must provide an alternative model which deduces the old and the new data; simply pointing out that some data not considered by the original author remains unexplained is not an interesting contribution.

The third option is to show that a simpler theory suffices to derive the same data. We want to create the simplest possible theory for the largest possible dataset. Such “minimalist” concerns are part of the method and automatically taken into account. There is no need for a special minimalist program to explore such questions.

A proper justification does not show that the theory is true. It shows that if the theory were true, the data would follow. The skeptic can therefore either show that the data was incomplete, biased or otherwise in need of revision, or that there is a simpler theory which entails the same facts. These two forces push us towards a theory which is as simple as possible while explaining as much data as possible. That represents a limit of what science can hope to accomplish.

4. Data selection

Proper justification replaces intuitive arguments and opinions by rigorous and unambiguous proofs, but the results must be evaluated in the context of a dataset. A dataset containing one expression can support only meaningless justifications, although technically it does support them. Likewise, a dataset containing a hodgepodge of expressions with no clear criteria for selection cannot produce convincing justifications, since the data could have been selected specifically for the purpose of supporting the theory. The data must therefore be selected systematically.

Linguistic research topics are usually thematized under labels such as “topic and focus interpretations in Finnish,” “Wh-movement” or “English passives.” These labels reflect the underlying linguistic phenomena one seeks to understand. Thus, a theory of any such topic presupposes that the linguistic phenomenon of interest is operationalized into a dataset. How to create a dataset given a topic of interest?

Ultimately, the stakeholders and experts must agree on a standardized dataset which contains the core examples. The standardized core examples should only contain essential data; it can be expanded into more challenging and peripheral domains by creating additional expanded datasets. For example, while the core examples operationalizing the phenomenon of English passive (just to mention one example) should probably contain object raising, agreement patterns, word order and *by*-phrases, additional data segments could expand to raising constructions, hyperraising, other examples of A-movement, adjectival passives, and so on. The crux of the matter is that the stakeholders must agree on what belongs to what dataset. Unfortunately as of this writing no such standardized datasets are available. Heuristics rules must substitute proper data selection.

The first, and the most exhaustive method, is to sample all logically relevant expressions. A study focusing on word order, for example, could compose the dataset by including all logically possible word order permutations (as in Brattico 2020). This is possible if the design of the study allows one

to construct the notion of ‘all logically relevant expressions’, and if that set is not too large. The justification can then be very strong.

The second selection method relies on grammatical variables and crosses them. In a (simplistic) study exploring finite agreement in English, for example, the grammatical variables could be (i) number and person of the subject; (ii) number and person on the finite verb, including the bare form; (iii) presence of an auxiliary, with all number and person forms; and others, depending on the size of the study (we can include two auxiliaries, passives, raising and others). The dataset is created by crossing the variables (examples of this method are Brattico 2023, Brattico & Chesi, 2021). The justification is weaker than the one produced by the exhaustive method, but can still be convincing. The problem is that the size of the dataset will quickly get out of hand as we increase the number of variables.

The third method is by relying on existing literature. In a study focusing on the island behavior of $\bar{A}$ -movement, for example, we could include all core constructions/configurations discussed in the existing literature (e.g., CED, Left Branch Condition, Wh-island, Anti-Agreement) and show that the theory handles them (e.g., Brattico 2022, 2025). This is the weakest method, but as things stand at present in many cases it is the only feasible alternative. Future studies will then complete the picture as more and more data will be brought under scrutiny, and possible omissions are recognized and fixed.

Whatever the method, the aim is to create datasets that do not contain unexplained gaps when evaluated against the linguistic phenomena they are designed to operationalize. And, as stated above, ultimately the whole community of stakeholders must participate in the creation of these sets.

The data is collected into a file (or files) and augmented with comments telling the reader what the data is, what the particular items illustrate, how the expressions are glossed and translated, in

general any information that might be useful for linguists who would like to work with the same phenomenon and data.

Properties of expressions create a special difficulty. If the theory predicts topic and focus readings, say, then the dataset must contain them. In the existing computational generative grammar literature (e.g., Brattico 2025) this problem has been tackled by expressing the properties of interest by using special notation that the theory replicates. Thus, each expression in the dataset comes with additional field or fields which provide the properties of interest, and which the theory then generates. For example, in the above mentioned study on binding the target expressions were paired with the attested binding configurations:

```
& 1. Initial binding condition tests (English)

# Condition C (1-3)

John admires Bill
|-> Binding: John[a] admire Bill[b]

he `s brother admires Bill
|-> Binding: he[b] brother[a] admire Bill[b,c]

Bill said that John admires Tim
|-> Binding: Bill[a] say John[b] admire Tim[c]
```

The theory was required to reproduce the information provided in the second line after each target expression (marked by special symbol `|->`).

The dataset format depends on whether we are working with a recognition or enumerative grammar. Recognition grammars work best with datasets containing a list of expressions and their properties (including unattested ones, marked specifically as such). See the above example. The script examines each item and verifies that the theory replicates its properties. The list format is easy to work with, and seems to match well with certain expectations of how a dataset should ‘look like’. Enumerative grammars, on the other hand, and because they generate expressions from initial parts, require that we pair the initial parts with target expressions. These two formats are related in such a way that each numeration-target(s) -pair corresponds to (an implicit) set of attested and unattested

expressions. Because the number of unattested combinations will usually be huge, full conversion can be infeasible and impractical. How to best combine the two formats will be left for future.

5. Multilingual datasets and explanatory adequacy

Several linguistic frameworks and theories study linguistic phenomena in a cross-linguistic way. Instead of describing and explaining English passives they are interested in the phenomenon of personal passive as it applies to potentially any language in the world. A linguistic theory which captures properties of several languages is said to reach the condition of “explanatory adequacy.” This raises several additional challenges for the computational generative grammar.

A cross-linguistic study, whatever its orientation or initial framework, presupposes a multilingual dataset. A multilingual dataset is like an ordinary monolingual dataset except that it contains data from several languages. Since speakers of different languages differ from each other, we cannot hope to use the exact same grammar for processing the whole dataset. Minimally each expression in language L must be processed by using the lexicon of L.

Human linguistic behavior can be decomposed into two factors: the innate cognitive architecture that all humans share as a universal species-wide property, and variation which results from innate and experience-based differences between individuals. Apart from severe pathologies there is no evidence that genetic differences play a role in the constitution of languages, hence we are left with the innate/universal part U and experience E.¹⁶ Differences between languages or more generally differences between groups of speakers are explained by relying on E; similarities by U. For example, all language-specific lexicons belong to E.

Let us assume that the linguistic items in the dataset are processed by a *speaker model*, an idealized model of a group of speakers. The grouping criteria can be based on language, dialect, age, pathology and so on. Suppose we construct the speaker models on the basis of language. A speaker

¹⁶ The term Universal Grammar (UG) is often used for the innate/universal part, but the term is usually understood as referring to the language-specific universal cognitive system, hence it is only part of what is contained in U.

model is constituted by U and E. The speaker model is created run-time (meaning ‘during the execution of the script’) by combining the universal properties with the variable parts, the latter depending on the languages detected in the dataset. The corresponding expressions are then processed by the respective speaker models. What is in the variable part depends on the theory, but minimally the two speaker models will differ in terms of their open-ended (and functional) lexicons: *Finnish speaker model* = U + Finnish lexicon, *English speaker model* = U + English lexicon. If languages differ in terms of a finite number of parameters, then the parameter settings too must be stored and provided for the speaker models. If the languages differ in terms of grammatical rules, then the rules must be provided in connection with the parameters and lexicons.

How much of the human linguistic behavior can be attributed to innate factors and to experience is an empirical question. To study such questions, some monolingual datasets will be expanded into multilingual ones.¹⁷ Over time the multilingual datasets will contain expressions from a wide variety of languages such that the sample is more or less representative of the world’s languages. Suppose we have such representative datasets available, and that the stakeholders agree that they constitute a useful starting point for multilingual theorizing. Then at one end we will have theories which minimize the universal part, perhaps to a point at which all languages are based on fundamentally different grammar models. If the model, furthermore, is observationally adequate against the multilingual dataset, it shows that an empiricist model suffices to describe and explain the data. A model of this type supports the empiricist viewpoint. Another researcher can try to simplify it by replacing the language-specific components with universal or near-universal rules, relying on U that all languages share. If the reduction succeeds, the model becomes simpler. The experience-based hypothesis gets replaced by an alternative in which the boundary between innate and learned components of language shifts towards the former.

17 This does not remove the need for large language-specific datasets which support detailed modeling of individual languages.

The innate-learning -debate will eventually get solved by finding the optimal spot between U and E such that the theory explains the widest multilingual dataset by fewest possible assumptions. At this point the theory reaches the condition of explanatory adequacy.

6. Linguistic levels

The purpose of the derivational search function is to define the notion of ‘possible derivation’, given a grammar. It was assumed that there is a set of linguistic operations OP which are applied to linguistic representations such as phrase structure trees and, ultimately, to the primitive items of the grammar such as words. A complication is that empirically plausible linguistic theories typically posit several linguistic levels, not just one. Morphology, syntax and semantics could each operate with their own primitives and rules of combination. In many theories syntax itself is divided into several levels such as surface and deep structures. It is not necessarily obvious how multilevel architectures map into the derivational search function provided in Section 3.

The linguistic levels posited by any remotely plausible theory must stand in systematic relations with each other. The nature of these mappings determines how the levels are embedded within the derivational search function. If the mappings are part of the operations OP, then the derivational search function must have access to the mapping operations OP^M . For example, OP_1^M could contain transformations that apply to finished phrase structure representations and produce variations of them. An alternative is that the mappings are implemented by interpreting the output of the syntactic engine, thus by separate modules or extra-linguistic systems which assume the output of the derivational search function as input and produce outputs corresponding to the separate ‘linguistic levels’ such as semantic interpretation or phonological spell-out. The latter matches with the standard Y-architecture used as a starting point in many generative theories today. The choice between these two approaches is empirical. Because interpretative rules map derivational outputs with representations in some other domain, they ‘decorate’ derivations with additional properties.

The derivational search function, in turn, generates additional branching points and enlarges the derivational space.

For example, something like the Finnish vowel harmony rule is perhaps best modeled in an interpretative component because it maps inputs in some morphophonological format into concrete phonological outputs (e.g., *talo-ssA* => /talossa/, *väri-ssA* => /värissä/). Each morphological representation is decorated with an additional property that gets matched against the phonological dataset. On the other hand, a *wh*-movement rule which preposes English interrogative pronouns could be best formulated as a transformational mapping rule, because there are echo-interpretations in which the rule does not apply. If the derivational search function has access to such rule, it generates both variants.

Some grammatical theories derive expressions from complex lexical entries which regulate syntactic construction and are spawned by a separate nonsyntactic lexical module regulated by its own rules of composition (so-called “lexicalist” theories). In a study focusing on syntax, we can provide these complex lexical items manually as they enter into syntactic derivations implemented by the derivational search function. Under this theory much of phrasal syntax originates in lexical information. Ultimately, however, we must provide the composition rules for the complex lexical items and test the theory with derivational search. We can then either (i) create a separate derivational search function for the lexical items, which creates inputs for phrasal syntax, or (ii) add the lexical composition rules to the one derivational search function deriving phrasal syntax. Options (i-ii) are notational variants, so the choice depends on practical concerns.

Morphology and word-formation are a much more contentious issue. A computational generative grammar model generates linearized sequences of phonological words separated by word boundaries. The individual morphemes comprising phonological words are separated by morpheme- and clitic-boundaries, perhaps others. Since word-formation is productive, a derivational search function must be involved. It could involve a separate lexical system, phrasal syntax and/or postsyntactic operations. The problem, however, is that it is not obvious that word-

formation belongs to any one component. Given the complex empirical issues involved, it seems that very large datasets will be required to settle these issues with any confidence.

7. Concluding thoughts

Computational generative grammars are theoretical constructs designed to deduce data. As such, they belong to theoretical linguistics. As in other advanced sciences it is likely that a certain division of labor will emerge that separates empirical and applied linguistics from theoretical linguistics, such that the former is specialized in dataset creation (data collection, experimentation, verification, dataset design), the latter in rigorous theorizing. Application of the computational techniques is to a certain extent a specialized field that requires special competences such as programming skill, whereas data collection might require extensive field and laboratory work.

Certain additional benefits of the computational generative grammar framework merit brief mention. One is, it becomes possible to calculate huge datasets. The calculations are also reliable. Because the calculations are both efficient and reliable, it becomes possible to test theoretical ideas by creating a script that performs the calculations for potentially large datasets automatically. The testing protocol is fast, reliable, efficient – multiple times more efficient and reliable than the old paper-and-pencil method.

Pauli Brattico

pauli.j.brattico@gmail.com

References

Brattico, P. (2019). *Computational implementation of a linear phase parser. Framework and technical documentation (version 20.1)*. IUSS. Retrieved 14.8.2026 from <https://github.com/pajubrat/parser-grammar/blob/master/docs/Old%20documentation/>

Computational%20implementation%20of%20a%20linear%20phase%20parser%20(version%202020.1).pdf

Brattico, P. (2020). Finnish word order: does comprehension matter? *Nordic Journal of Linguistics*, 44(1), 38–70.

Brattico, P. (2021). Computational linguistics as natural science. *Lingbuzz/005796*.
https://ling.auf.net/lingbuzz/005796?_s=MUzSsZPWMdmoX8nI&_k=3e1ty8tPIVI3hVmY.

Brattico, P. (2021). Computation and the justification of grammars. *Finno-Ugric Languages and Linguistics*, 10(1–2), 51–74.

Brattico, P. (2022). Predicate clefting and long head movement in Finnish. *Linguistic Inquiry*, 54(4), 663–692.

Brattico, P. (2023). Computational analysis of Finnish nonfinite clauses. *Nordic Journal of Linguistics*, 1–40.

Brattico, P. (2025). Binding in Finnish and the language-cognition interface. *Journal of Uralic Languages*, 4, 271–321.

Brattico, P., & Chesi, C. (2020). A top-down, parser-friendly approach to operator movement and pied-piping. *Lingua*, 233, 102760.

Appendix A

Derivational search function in Python

This appendix describes the implementation of the derivational search function in Python, as it was implemented in Brattico (submitted). The function is as follows:

```
638     def derivational_search_function(self, sWM):
639         """Derivational search function"""
640         if self.derivation_is_complete(sWM):
641             self.process_final_output(sWM)
642         else:
643             for Preconditions, OP, n, name in self.syntactic_operations:
644                 for SO in itertools.permutations(sWM, n):
645                     if Preconditions(*SO):
646                         PhraseStructure.logging_report += f'\t{name.upper()}({mngmnt.print_lst(SO)}) '
647                         sWM_, SO_ = mngmnt.sWMCopy(sWM, SO)
648                         new_sWM = sWM_ | mngmnt.tset(OP(*SO_))
649                         self.consume_resource(new_sWM, sWM)
650                         self.derivational_search_function(new_sWM)
651     mngmnt.log_file.write('.')
```

Symbol `sWM` (line 638) denotes the syntactic working memory, which is a set of linguistic phrase structure objects. The function gets this set as its input parameter. Lines 640-641 check that the derivation is complete, and executes functions that have to do with post-syntactic processing such as linearization. They create the final string that gets matched with the target sentences in the dataset. If the derivation can continue, we go through all derivational operations OP_n of the grammar as they are provided in the list `syntactic_operations` (line 643) and all n -tuples $(x_1, \dots, x_n)$ of the objects in the `sWM` (line 644). If the preconditions of OP_n are satisfied by $(x_1, \dots, x_n)$ (line 645), a new syntactic working memory (`new_sWM`) is created by replacing $x_1, \dots, x_n$ by $OP_n(x_1, \dots, x_n)$ (lines 647-648) and calling the function recursively with the new syntactic working memory configuration (line 650). Lines 646 and 649 implement logging (see the main text, Section 3 for an example of what they produce). The syntactic operations are in a list which contains the names of the functions implementing the operations, such as Merge and Adjoin in this study.

Appendix B

Phrase structure trees and grammatical operations in Python

The derivational search function described in Section 3 of the main article and implemented in Python in Appendix A presupposes that we have a formalized system of phrase structure representations and grammatical operations at hand. The former are stored in the syntactic working memory and become the target of grammatical operations; the latter are in the list containing all syntactic operations. Starting from the first, it is perhaps best to define the phrase structure system in its own class so that it can be modified freely as required by the theory and empirical observations. The class contains properties related to the phrase structure geometry and other properties, for example:

```
59 class PhraseStructure:
60     """Simple asymmetric binary-branching bare phrase structure formalism"""
61     logging = None           # class variable: whether logging is on c
62     chain_index = 1         # class variable: creates chain indexes
63     logging_report = ''     # class variable: used to store logging i
64     logging_report_detailed = ''
65
66     def __init__(self, X=None, Y=None, **kwargs):
67         self.const = (X, Y) # Constituents are repres
68         self.features = set() # Features (set of string
69         self.mother = None    # Phrase structure geomet
70         if X:                 # ...
71             X.mother = self   # ...
72         if Y:                 # ...
73             Y.mother = self   # ...
74         self.zero = False     # Whether the object is a
75         self.elliptic = False # Ellipsis due to copying
76         self.copied = None    # Copy link
77
78         # Input arguments
79         if 'features' in kwargs.keys():
80             self.add_features(kwargs['features'])
81
82         # Auxiliary properties not part of the empirical theory
83         self.adjuncts = set() # Stores the set of adjuncts adjoin
84         self.adjunct = False  # Auxiliary assumption, not part of
85         self.isomapping = None # Auxiliary structure for backtracki
86         self.chain_index = 0  # Initial chain index
87         self.phonological_exponent = '' # Phonological exponent of the const
```

Class variables (lines 61-64) are used for logging operations on phrase structure objects. Daughter constituents are provided as tuples (line 67) but this is only one of many options. Other possibilities

are list, dictionary, independent variables (e.g., `self.left`, `self.right`).¹⁸ Each constituent can have one mother (line 69). Each constituent may have a set of features (line 69) that can be provided as an input keyword argument (lines 78-80). Lines 74-76 provide properties that have to do with the particular empirical theory assumed in this study and will be ignored here. The `init`-function gets the daughter constituents `X` and `Y` as input together with the keyword arguments, if any, hence it is possible to implement a simple binary Merge-operation by `Z = PhraseStructure(X, Y)` which is the same as `Z = Merge(X, Y)`.

Elementary operations defined for phrase structure constituents can be provided in this class. For example:

```
109     def Merge(X, Y):
110         """Standard Merge"""
111         return PhraseStructure(X, Y)
```

These operations can then be referred to in the list containing all grammatical operations, which will automatically include them into the derivational search (Appendix A).

18 The choice should align as closely as possible with the empirical theory. The primitive data-structures contained here can be encapsulated inside an additional abstraction layer which corresponds better to the theory.